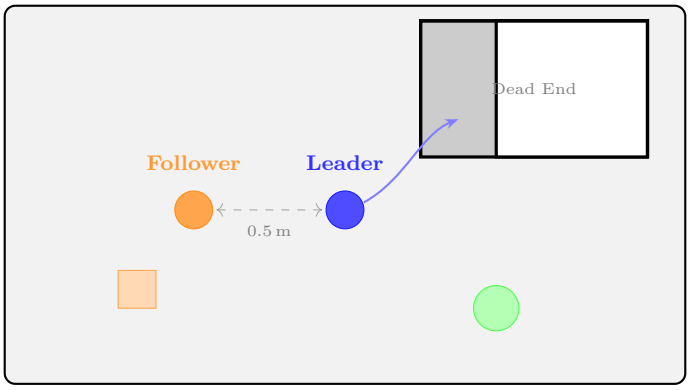
Model-Driven Engineering Lab

Leader-Follower Multi-Robot System

with ROS2, Gazebo, and Uppaal



Course: Model-Driven Engineering
Hardware: 2× TurtleBot3 Burger (ROS2 Humble)
Tools: Uppaal 5.x · ITEMIS Create · Gazebo Classic 11

Contents

1 Introduction and Scenario **2**
- 1.1 Motivation . . . 2
- 1.2 The Scenario . . . 2
- 1.3 Edge Cases to Handle . . . 2

2 Background: Model-Driven Engineering **4**
- 2.1 The MDE Workflow . . . 4
- 2.2 Tools Used . . . 4
- 2.3 Repository Structure . . . 4

3 Stage 1 — Requirements Analysis **5**
- 3.1 System Requirements . . . 5
  - 3.1.1 Functional Requirements . . . 5
  - 3.1.2 Safety Requirements . . . 5

4 Stage 2 — State Machine Modelling (ITEMIS Create) **6**

5 Stage 3 — Timed Automata (Uppaal) **6**
- 5.1 What is a Timed Automaton? . . . 6
- 5.2 Query Language . . . 6

6 Stage 4 — Property Verification Summary **8**

7 Stage 5 — Code Skeleton and Implementation **9**
- 7.1 Architecture Overview . . . 9

8 Stage 6 — Simulation Testing (Gazebo) **10**
- 8.1 Launching the Simulation . . . 10
- 8.2 Expected Startup Sequence . . . 10

9 Stage 7 — Real Robot Deployment (Bonus) **12**
- 9.1 Network Setup . . . 12
- 9.2 Per-Robot Bring-Up . . . 12

1 Introduction and Scenario

1.1 Motivation

Modern software systems are rarely designed purely from intuition. *Model-Driven Engineering* (MDE) is a software development paradigm in which models—not code—are the primary artifacts of a project. By formalising behaviour through diagrams, timed automata, and verified models *before* writing a single line of executable code, we gain:

- Early detection of design flaws (deadlocks, livelocks, unreachable states);
- Unambiguous communication of system behaviour across the team;
- A clear, traceable path from abstract specification to concrete implementation;
- Confidence that the final system satisfies stated requirements.

In this lab you will follow the full MDE workflow for a real robotic application: you will *model*, *verify*, *generate*, *implement*, *simulate*, and finally *deploy* a cooperative multi-robot system.

1.2 The Scenario

You are given two TurtleBot3 Burger ground robots operating in an indoor environment. One robot acts as the **Leader**: it explores the environment using Simultaneous Localisation and Mapping (SLAM), building a map as it goes. The second robot acts as the **Follower**: it tracks the leader and stays approximately 0.5 m behind it at all times.

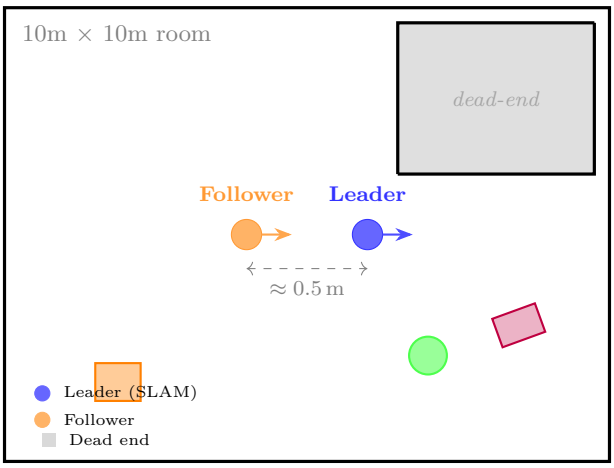


Figure 1: Top-down view of the exploration environment. The leader navigates into the dead-end corridor (NE quadrant); the follower must coordinate to resolve the situation.

1.3 Edge Cases to Handle

Real cooperative robot systems must be robust. The following *edge cases* are part of the formal specification and must be handled by your implementation:

EC1. Follower too far. The leader starts moving but the follower has not yet reached the correct following distance. The leader must wait while the follower approaches.

EC2. Leader hits a dead end. The leader detects that all forward paths are blocked by obstacles. It signals the follower to back up, reverses out, and resumes exploration. The follower temporarily drives forward to keep the path clear.

EC3. Follower loses the leader. Signal timeout or excessive distance causes the follower to lose track. The follower searches using the last known position; the leader halts and broadcasts its location until the follower recovers.

EC4. Both robots stuck (mutual wait). If both robots are simultaneously waiting for the other, the system must eventually time out and recover — not deadlock indefinitely.

2 Background: Model-Driven Engineering

2.1 The MDE Workflow

In MDE, a project proceeds through a series of increasingly concrete models, each derived from the previous one through *transformations*. Figure 2 shows the workflow for this lab.

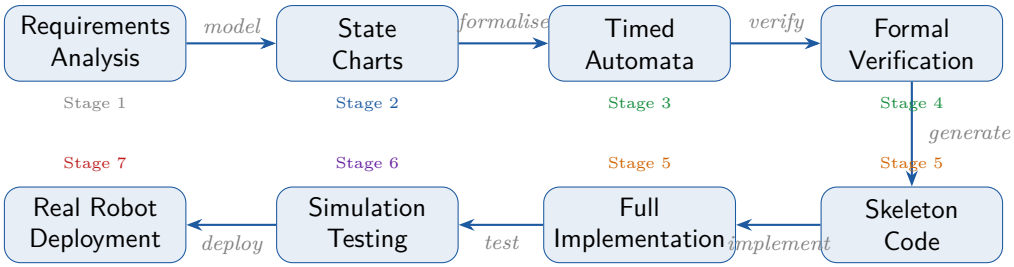


Figure 2: The MDE workflow followed in this lab. Each arrow represents a transformation.

2.2 Tools Used

Tool	Purpose	Stage
ITEMIS Create	Graphical statechart editor	2
Uppaal 5.x	Timed automata editor + model checker (free, uppaal.org)	3–4
ROS2 Humble	Robot operating system middleware	5–7
Gazebo Classic 11	Physics-based simulation	6
Nav2	Navigation stack (path planning)	5–7
slam_toolbox	SLAM for the leader robot	5–7
RViz2	Visualisation of map, robots, markers	6–7

2.3 Repository Structure

The following folder structure is a proposal for organising your project files.:

```
leader_follower/
  models/
    statecharts/
      leader_statechart.ysc        # Leader ITEMIS Create model
      follower_statechart.ysc      # Follower ITEMIS Create model
    uppaal/
      leader_follower.xml          # Uppaal model
      leader_follower.q            # Verification queries
  src/
    leader_follower_msgs/          # Custom ROS2 messages
    simulation/                    # Gazebo world, launch files, RViz config
    leader_node/
      leader_node/
        leader_node.py
    follower_node/
      follower_node/
        follower_node.py
    visualization_node/            # visualises robots in RViz2
```

3 Stage 1 — Requirements Analysis

Goal of this Stage

Understand the system requirements before writing any model or code.

3.1 System Requirements

The system must satisfy the following requirements.

3.1.1 Functional Requirements

FR1. The leader robot shall explore the environment autonomously using SLAM.

FR2. The follower robot shall maintain a distance of 0.5 ± 0.2 m behind the leader.

FR3. If the follower's distance exceeds 1.5 m, the leader shall pause and wait.

FR4. If the leader detects a dead end (all forward paths blocked), it shall signal the follower and execute a reversal manoeuvre.

FR5. If the follower loses contact with the leader (no status message for > 5 s), the follower shall execute a search procedure and the leader shall halt.

FR6. Both robots shall recover from every edge case without operator intervention, within the time bounds specified in the timed automata model.

3.1.2 Safety Requirements

SR1. The system shall *never* deadlock — at least one robot must always be able to make progress.

SR2. The follower shall not drive closer than 0.2 m to any obstacle.

SR3. The maximum time spent in any waiting state shall be bounded (see Table 1).

Table 1: State timeout bounds (encoded as clock invariants in the Uppaal model).

State	Robot	Max duration
`WAITING_FOR_FOLLOWER`	Leader	60 s
`DEAD_END`	Leader	15 s
`WAITING_FOR_LOST_FOLLOWER`	Leader	120 s
`APPROACHING`	Follower	60 s
`SEARCHING_FOR_LEADER`	Follower	90 s

Exercise 3.1 — Requirements Review

Before proceeding, answer the following questions in your lab report:

1. Which requirements are *safety* properties ("bad things never happen")? Which are *liveness* properties ("good things eventually happen")?
2. Describe in your own words what edge case EC4 (mutual wait) means in terms of robot behaviour. What would happen without the timeout bounds?
3. List any additional edge case you think the specification is missing. Justify your answer.

4 Stage 2 — State Machine Modelling (ITEMIS Create)

Goal of this Stage

Create precise, readable statechart models for both robots. Statecharts are the bridge between informal requirements and formal timed automata.

Exercise 4.1 — Statechart Creation

1. Model the leader and follower behaviours as statecharts in ITEMIS Create, following the provided requirements and edge cases.
2. Extend your statecharts to cover a scenario not yet modelled (e.g. battery low, sensor fault). Justify your choice.

5 Stage 3 — Timed Automata (Uppaal)

Goal of this Stage

Translate the statecharts into Uppaal timed automata, which allow formal verification. The key difference from statecharts: every state can have a *clock invariant* ("this state is exited within x seconds") and transitions can have *guards* (conditions on clocks and variables).

5.1 What is a Timed Automaton?

A Timed Automaton (TA) extends a finite-state automaton with:

- **Clocks** $x, y, \ldots \in \mathbb{R}_{\geq 0}$ — real-valued timers that increment continuously.
- **Clock invariants** on locations: e.g. $x \leq 15$ means "leave this state within 15 seconds."
- **Clock guards** on transitions: e.g. $x \geq 5$ enables a transition only after 5 seconds.
- **Resets**: $x := 0$ resets a clock on a transition.

A **network of timed automata** (NTA) is a set of TA running concurrently, with synchronising via channels: `signal!` (sender) pairs with `signal?` (receiver).

Here are examples of global channel declarations you might find useful when designing your own model. The list is not exhaustive; you may need to add more channels for additional scenarios or edge cases.

Listing 1: Global declarations in leader_follower.xml.

```
chan follower_ready;           // Follower -> Leader: in position
chan leader_start;             // Leader -> Follower: begin
broadcast chan dead_end_alert; // Leader -> Follower: dead end!
chan follower_backed_up;        // Follower -> Leader: backed up
```

5.2 Query Language

Uppaal uses a subset of Computation Tree Logic (CTL):

Syntax	Reading	Type
`A[] p`	p always holds on all paths	safety
`E<> p`	p eventually holds on some path	reachability
`A<> p`	p eventually holds on all paths	(strong) liveness
`E[] p`	p always holds on some path	—
`p -> q`	whenever p, eventually q (leads-to)	liveness

The special predicate `deadlock` is true in a state where no transition is enabled (even after arbitrary time passage).

Exercise 5.1 — Loading and Running Queries

1. Model the leader and follower behaviours as timed automata in Uppaal, following the provided requirements and edge cases.
2. Run all 14 queries below. Record your results in the following table (your model needs to take care of the mentioned signals and timeouts for the queries to pass):

#	Query	Type	Expected	Result
1	`A[] not deadlock`	safety	SAT	
2	`E<> (Leader.EXPLORING and Follower.FOLLOWING)`	reach.	SAT	
3	`Follower.APPROACHING --> Follower.FOLLOWING`	liveness	SAT	
4	`Leader.DEAD_END --> Leader.EXPLORING`	liveness	SAT	
5	`Follower.SEARCHING_FOR_LEADER --> Follower.APPROACHING`	liveness	SAT	
6	`E<> Leader.DEAD_END`	reach.	SAT	
7	`E<> Follower.LEADING_TEMPORARILY`	reach.	SAT	
8	no mutual timeout (clock bound)	safety	SAT	
9	`A[] (WAIT_FOLLOWER` $\Rightarrow$ `t` $\leq$ `60)`	safety	SAT	
10	`A[] (SEARCHING` $\Rightarrow$ `t` $\leq$ `90)`	safety	SAT	
11	progress from EXPLORING	liveness	SAT	
12	`A<> (Leader.EXPLORING and Follower.FOLLOWING)`	univ. live.	?	
13	mutual exclusion of conflicting states	safety	SAT	
14	follower leading $\Rightarrow$ leader reversing/exploring	safety	SAT	

Exercise 5.2 — Analysing Query 12

Query 12 (`A<>` universal liveness) is marked with "?" because it may *not* be satisfied.

1. Run query 12. If it fails, Uppaal will show a *counter-example trace*. Use the simulator to step through it and identify which states are reached.
2. Explain in your own words why `A<>` is harder to satisfy than `E<>`.
3. A failure can be modelled by an explicit `ERROR` state. Add an `ERROR` state to both the `Leader` and `Follower` templates and modify query 12 to *assume* neither robot is in `ERROR` and re-verify.
4. Add a new property of your own that is meaningful for this system (e.g. "after a dead end, reversal completes within 30 seconds"). Write the query and verify it. Explain the result.

Exercise 5.3 — Extending the Uppaal Model

1. Model distance changes by cycling through values 0–2 deterministically in an `Environment` automaton.
2. Modify it so that the distance can also jump directly to a LOST state at any time, simulating sudden signal loss. Re-verify the deadlock-freedom query.
3. Add a `BatteryMonitor` template that:

- maintains an integer variable `battery` ($100 \to 0$) that decreases over time;
- sends an `emergency_stop` broadcast when `battery == 0`;
- resets when `emergency_stop` is received (robot returns to dock).

Write at least two new queries for the extended model.

6 Stage 4 — Property Verification Summary

Goal of this Stage

Consolidate your verification results and argue formally about key system properties in your presentation.

Exercise 6.1 — Verification Report

For the presentation, prepare a concise report summarising your verification results.

1. States the three most important properties for this system and justifies why.
2. For each property, gives:
   - The informal English description,
   - The formal Uppaal query,
   - The verification result (satisfied / not satisfied),
   - A brief argument for *why* it holds (or what assumption is needed).

7 Stage 5 — Code Skeleton and Implementation

Goal of this Stage

Translate the verified formal model into executable ROS2 Python code.

7.1 Architecture Overview

The system can run as five cooperating ROS2 nodes:

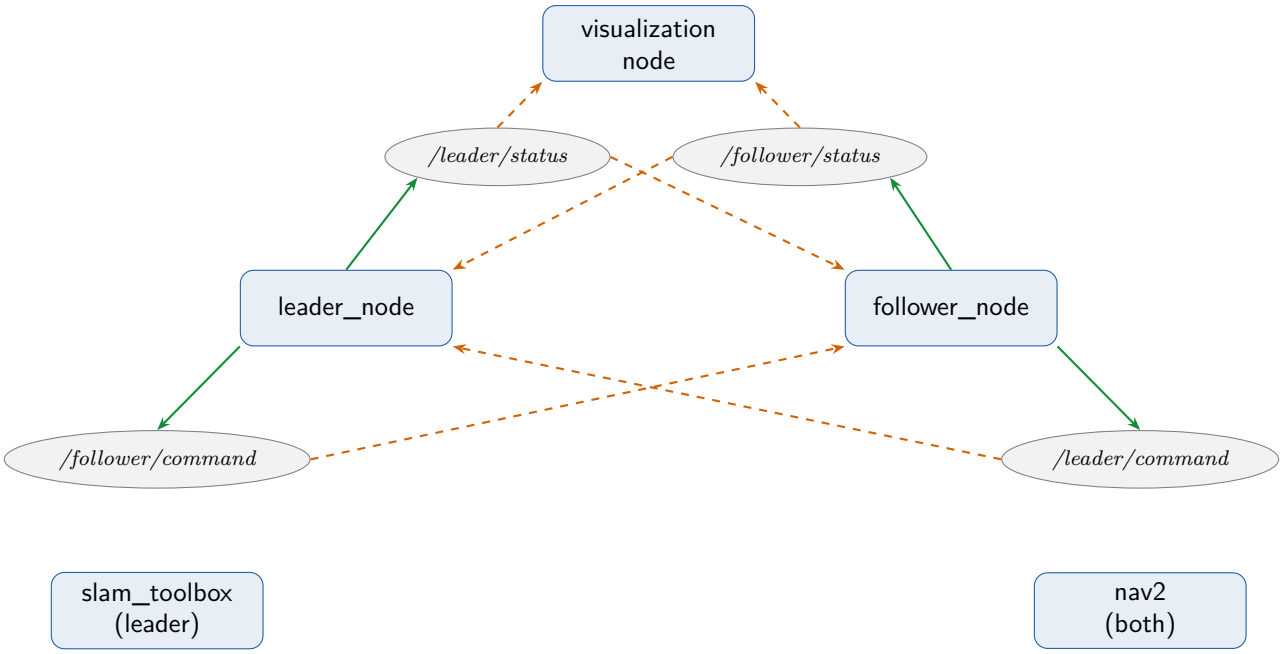


Figure 3: ROS2 node communication graph. Green solid = publish; orange dashed = subscribe.

Exercise 7.1 — Implementing the Nodes

1. Implement the `leader_node` and `follower_node` state machine logic in Python, following the verified model.
2. Implement the `visualization_node` to display the positions and states of both robots in RViz2 using markers.
3. Integrate the `slam_toolbox` for the leader and `nav2` for both robots to enable autonomous navigation and obstacle avoidance.

8 Stage 6 — Simulation Testing (Gazebo)

Goal of this Stage

Validate your implementation in the Gazebo simulation environment before touching real hardware.

8.1 Launching the Simulation

```
# Terminal 1: start simulation (Gazebo + Nav2 + SLAM + all nodes)
source install/setup.bash
ros2 launch simulation simulation.launch.py

# Terminal 2: monitor robot states
ros2 topic echo /leader/status --field state_name
ros2 topic echo /follower/status --field state_name

# Terminal 3: watch coordination commands
ros2 topic echo /follower/command
ros2 topic echo /leader/command
```

8.2 Expected Startup Sequence

1. Gazebo loads an exploration world (build a sample world); both robots spawn.
2. Leader enters `INIT`, follower enters `INIT`.
3. Follower transitions to `APPROACHING`, drives toward leader.
4. Follower sends `CMD_READY`; leader transitions to `EXPLORING`.
5. Both enter cooperative mode: `EXPLORING` / `FOLLOWING`.
6. Leader navigates into a dead-end corridor.
7. Leader detects dead end, sends `CMD_DEAD_END`.
8. Follower transitions to `BACKING_UP`, then `LEADING_TEMPORARILY`.
9. Leader reverses; clears the corridor; sends `CMD_RESUME`.
10. Both return to `EXPLORING` / `FOLLOWING`.

Exercise 8.1 — Observe and Document Normal Operation

1. Build a simple Gazebo world with an open area and a dead-end corridor.
2. Record a sequence of state transitions from the terminal output. Do they match the expected startup sequence above?
3. Measure the time from simulation start until both robots first reach cooperative mode (`EXPLORING`/`FOLLOWING`). Compare with the timeout bounds in Table 1.

Exercise 8.2 — Test Edge Case EC2: Dead End Recovery

1. Wait until the leader naturally navigates into the dead-end corridor, *or* use teleop to manually drive the leader into the corridor:

   ```
   ros2 run turtlebot3_teleop teleop_keyboard \
     --ros-args --remap /cmd_vel:=/leader/cmd_vel
   ```

2. Observe the state transitions in both robots. Record the exact sequence of

`state_name` values published on `/leader/status` and `/follower/status`.

3. Does the dead-end recovery protocol complete within the timeout specified in Table 1? Time it.

Exercise 8.3 — Test Edge Case EC3: Follower Lost

1. While both robots are in cooperative mode, kill the follower node:

```
# Find the follower node's PID and kill it
ros2 node list
kill $(ros2 node info /follower/follower_node | grep Pid | awk '{
    print $2}')
# Or simply Ctrl+C in the terminal running follower_node
```

2. Observe: the leader should transition to `WAITING_FOR_LOST_FOLLOWER` within 5 seconds and begin broadcasting `CMD_I_AM_HERE`.
3. Restart the follower node:

```
ros2 run follower_node follower_node
```

4. Verify that the follower re-enters `APPROACHING` → `FOLLOWING` and the leader resumes `EXPLORING`.
5. Document the recovery time (seconds from follower restart to cooperative mode).

9 Stage 7 — Real Robot Deployment (Bonus)

Goal of this Stage

Deploy the validated simulation code on real TurtleBot3 Burger hardware. **This stage is optional**.

9.1 Network Setup

Both robots and the workstation must share the same ROS2 DDS domain:

```
# On ALL devices (workstation, leader Pi, follower Pi):
export ROS_DOMAIN_ID=42
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp

# Verify discovery:
ros2 node list    # should show nodes from all devices
```

9.2 Per-Robot Bring-Up

```
# On Leader Raspberry Pi:
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_bringup robot.launch.py

# On Follower Raspberry Pi:
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_bringup robot.launch.py

# On Workstation:
ros2 launch simulation real_robot.launch.py
```

Exercise 9.1 — Real Robot Validation

1. Place the robots in the correct start positions (0.5 m apart, facing the same direction). Observe the initial approach sequence.
2. Demonstrate the dead-end recovery by placing a large box in front of the leader while both robots are in cooperative mode.
3. Record a video of the full demo (initial approach + exploration + at least one edge case recovery).